
Análisis

1. ¿Qué propiedades y métodos tendrá cada clase?

Para realizar este programa se utilizarán cuatro clases: PuntoAcceso, Visitante, Parque y Principal.

La clase PuntoAcceso servirá para guardar la información de cada punto de acceso del parque. Tendrá propiedades como el código, nombre, ubicación, capacidad máxima por hora y estado. También tendrá un constructor para darle los valores iniciales, getters para obtener la información y setters para modificar la capacidad y el estado. Además, tendrá un método toString() para poder mostrar la información completa del punto de acceso.

La clase Visitante representará a cada persona que ingrese al parque. Tendrá como propiedades el código de entrada, nombre, edad, cantidad de atracciones visitadas y puntos acumulados. También tendrá un constructor, getters, setters para modificar algunos datos y un método toString() para mostrar la información del visitante.

La clase Parque será la encargada de guardar toda la información del parque y administrar los puntos de acceso y los visitantes. Tendrá el nombre del parque, código de identificación, nombre del encargado, un arreglo para guardar los puntos de acceso y un ArrayList para guardar los visitantes. Esta clase tendrá métodos para habilitar, consultar, modificar y cerrar los puntos de acceso. También tendrá métodos para registrar, buscar, modificar y eliminar visitantes. Además, permitirá realizar los cálculos que se solicitan en el reporte.

Por último, la clase Principal tendrá el método main() y será la encargada de mostrar el menú, pedir los datos al usuario utilizando Scanner y ejecutar las diferentes opciones. También servirá para manejar los errores que puedan ocurrir mientras el usuario utiliza el programa.

2. ¿Qué tipo deben tener las propiedades y métodos de cada clase?

Las propiedades como código, nombre, ubicación, estado y nombre del encargado serán de tipo String, ya que almacenarán texto.

Las propiedades que representan cantidades, como la capacidad máxima por hora, edad, cantidad de atracciones visitadas y puntos acumulados, serán de tipo int.

En la clase Parque se utilizará un arreglo de tipo PuntoAcceso[] para guardar los puntos de acceso. También se utilizará un ArrayList<Visitante> para guardar a los visitantes.

Los getters retornarán el tipo de dato que corresponda a cada propiedad. Los setters y algunos métodos que solo realizan una acción serán de tipo void. Los métodos que busquen un visitante o un punto de acceso podrán retornar el objeto encontrado y si no existe, retornarán null. El método para eliminar un visitante puede retornar un boolean, los métodos de conteo retornarán un int y el promedio de edad retornará un double. El método toString() retornará un String.

3. ¿Cuál de las propiedades debe implementarse utilizando un arreglo básico?

La propiedad que utilizará un arreglo básico será `puntosAcceso` de la clase `Parque`. Este arreglo guardará objetos de tipo `PuntoAcceso` y tendrá un tamaño fijo de cinco posiciones, porque el parque solamente puede tener un máximo de cinco puntos de acceso.

Al crear el arreglo, todas las posiciones estarán vacías y tendrán el valor `null`. Cuando se habilite un punto de acceso, se guardará un objeto en una de esas posiciones. Si una posición tiene `null`, significa que todavía está disponible.

4. ¿Cuál de las propiedades debe implementarse utilizando un ArrayList?

La propiedad que utilizará un `ArrayList` será `visitantes` de la clase `Parque`. Este será de tipo `ArrayList<Visitante>` y almacenará los objetos de los visitantes registrados.

Se utilizará un `ArrayList` porque no se sabe cuántas personas ingresarán al parque, por lo que también permitirá buscar, modificar y eliminar visitantes cuando sea necesario.

5. ¿Cuáles deben ser los modificadores de visibilidad?

Todas las propiedades de las clases serán `private`, ya que así se protege la información de los objetos y no se puede modificar directamente desde otra clase.

Los constructores, getters, setters y métodos que se necesiten utilizar desde otras clases serán `public`.

El método principal también será público y tendrá la forma `public static void main(String[] args)`, ya que será el método que iniciará el programa.

6. ¿Qué parámetros serán requeridos por los métodos?

Los constructores necesitarán los datos necesarios para crear cada objeto, por ejemplo, el constructor de `Parque` necesitará el nombre, código de identificación y nombre del encargado.

El constructor de `PuntoAcceso` necesitará el código, nombre, ubicación, capacidad máxima por hora y estado.

El constructor de `Visitante` necesitará el código de entrada, nombre, edad, cantidad de atracciones visitadas y puntos acumulados.

Otros métodos también necesitarán parámetros dependiendo de lo que hagan, para habilitar un punto de acceso se necesitará indicar la posición y el objeto que se quiere guardar.

7. ¿Cómo se proveerán valores iniciales y qué valores deberán validarse?

Los valores iniciales se proporcionarán por medio de los constructores, cuando se cree un objeto se ingresarán los datos necesarios y estos se guardarán en sus respectivas propiedades.

En el caso del `Parque`, también se creará el arreglo de cinco puntos de acceso y el `ArrayList` de visitantes.

Se deberán validar algunos datos antes de guardarlos. En PuntoAcceso, la capacidad máxima por hora deberá ser mayor que cero, por lo que si se ingresa una capacidad menor o igual a cero, se generará una `IllegalArgumentException`.

En Visitante, la edad deberá ser mayor que cero, así mismo la cantidad de atracciones visitadas y los puntos acumulados no podrán ser negativos, por lo que si se ingresa un valor incorrecto también se generará una `IllegalArgumentException`.

8. ¿Cómo se determinará si una posición del arreglo contiene un punto de acceso o contiene null?

Primero se deberá comprobar que la posición ingresada sea válida y que esté dentro de los límites del arreglo, como el arreglo tendrá cinco posiciones, solamente se podrán utilizar las posiciones del 0 al 4.

Después se revisará si esa posición contiene null. Si contiene null, significa que no hay ningún punto de acceso habilitado y que la posición está disponible, mientras que si contiene un objeto, significa que sí existe un punto de acceso en esa posición.

9. ¿Cómo se realizarán las operaciones de búsqueda, modificación y eliminación dentro del ArrayList?

Para buscar un visitante se recorrerá el ArrayList y se comparará el código ingresado con el código de cada visitante, por lo que, si se encuentra uno con el mismo código, se mostrará o se retornará ese visitante, mientras que, si no se encuentra, se indicará que el visitante no está registrado.

Para modificar un visitante, primero se buscará utilizando su código de entrada, por lo que, si existe, se podrán modificar sus datos utilizando los setters. Los nuevos datos deberán cumplir con las mismas validaciones que se utilizaron cuando se creó el visitante.

Para eliminar un visitante también se buscará dentro del ArrayList, por lo que cuando se encuentre el visitante correcto, se utilizará `remove()` para eliminarlo, si no existe ningún visitante con ese código, se mostrará un mensaje indicando que no fue encontrado.

10. ¿Qué situaciones del programa pueden producir excepciones?

Una situación que puede producir una excepción es cuando el programa solicita un número y el usuario ingresa texto, en este caso puede ocurrir una `InputMismatchException`, esta excepción se manejará utilizando try-catch para mostrar un mensaje y permitir que el usuario continúe utilizando el programa.

También se pueden producir excepciones cuando se ingresan valores que no cumplen con las reglas, por ejemplo, una capacidad menor o igual a cero, una edad menor o igual a cero o una cantidad negativa de atracciones o puntos, en estos casos se utilizará una `IllegalArgumentException`.

También se deberá tener cuidado con las posiciones inválidas del arreglo y con las posiciones que contienen null, por lo que antes de utilizar una posición se comprobará que sea válida y que contenga un objeto.

Los bloques try-catch se utilizarán principalmente en el menú y en las opciones donde el usuario ingrese datos, de esta manera, si ocurre un error, el programa no se cerrará y podrá continuar funcionando.

También se utilizará un bloque finally en alguna operación del programa, por lo que servirá para realizar una acción final sin importar si la operación se realizó correctamente o si ocurrió una excepción.